围棋 AI 大作业

> **作业说明文档**：docs/homework.pdf
>
> 本项目为题目一（围棋 AI）的实现框架。

围棋是一种古老的棋类游戏，蕴含着古人的智慧。2016 年，AlphaGo 击败人类顶尖棋手，标志着人工智能在博弈领域取得里程碑式的突破，其核心正是蒙特卡洛树搜索（MCTS）与深度神经网络的结合。

本题目要求实现一个基于蒙特卡洛树搜索的简易围棋 AI，能够与人类用户进行对弈。我们提供围棋的基本规则代码（包括落子合法性、提子、禁入点、终局判断等），你需要自行实现决策模块，并设计必要的图形化界面。

助教信息

- 席子恒：xizh21@mails.tsinghua.edu.cn
- 李自远：liziyuan22@mails.tsinghua.edu.cn

AI 辅助声明

本项目作业框架在 **Claude Code + GLM-5** 的辅助下完成编写。

项目结构

```
hw1/
├── docs/                  # 【文档】作业说明
│   └── homework.pdf       # 作业要求 PDF
│
├── dlgo/                  # 【已提供】围棋规则基础设施
│   ├── __init__.py        # 模块导出
│   ├── gotypes.py         # Player, Point 等基础类型
│   ├── goboard.py         # Board, GameState, Move 核心逻辑
│   ├── scoring.py         # 计分系统
│   └── zobrist.py         # Zobrist 哈希表
│
├── agents/                # 【学生实现】智能体算法
│   ├── __init__.py
│   ├── random_agent.py    # 第一小问：随机 AI
│   ├── mcts_agent.py      # 第二小问：MCTS AI
│   └── minimax_agent.py   # 第三小问：Minimax AI（选做）
│
├── play.py                # 命令行对弈脚本
└── README.md              # 本文件
```

作业要求

第一小问（必做）：随机 AI

要求：

- 熟悉给定规则代码或自行实现规则编写
- 在 5×5 棋盘上基于随机落子（但需满足规则）的方式实现一个基础的围棋 AI
- 验证规则的调用

实现文件： `agents/random_agent.py`

测试命令：

```
python play.py --agent1 random --agent2 random --size 5
```

第二小问（必做）：MCTS AI

要求：

- 在 5×5 棋盘上，实现基于标准 MCTS 算法的围棋 AI
- 算法包含**选择、扩展、模拟（随机走子走至终局）、反向传播**四个步骤
- 该 AI 需能根据当前棋盘状态，在合理时间（如 10s）内完成落子
- 与用户持续对弈
- **采取至少两种方法，尝试提升标准 MCTS 搜索效率**，例如：
  - 启发式走子策略（非完全随机）
  - 限制模拟深度（如 20-30 步）
  - 其他：RAVE、池势启发等

实现文件： `agents/mcts_agent.py`

需完成的核心方法：

1. `MCTSNode.best_child()` - UCT 选择公式
2. `MCTSNode.expand()` - 展开子节点
3. `MCTSNode.backup()` - 反向传播
4. `MCTSAgent.select_move()` - MCTS 主循环
5. `MCTSAgent._simulate()` - 随机模拟（含优化策略）

测试命令：

```
# MCTS vs 随机 AI
python play.py --agent1 mcts --agent2 random --size 5

# MCTS 对战 MCTS
python play.py --agent1 mcts --agent2 mcts --size 5 --games 10
```

第三小问（选做）：Minimax AI

要求：

- 实现基于极小化极大（minimax）搜索算法的围棋 AI
- 实现 Alpha-Beta 剪枝优化
- 与 MCTS 搜索对比

实现文件： `agents/minimax_agent.py`

需完成的核心方法：

1. `minimax()` - 基础递归算法
2. `alphabeta()` - Alpha-Beta 剪枝优化
3. `_default_evaluator()` - 局面评估函数
4. `GameResultCache.put()` - 置换表缓存

测试命令：

```
# Minimax vs 随机 AI
python play.py --agent1 minimax --agent2 random --size 5

# Minimax vs MCTS
python play.py --agent1 minimax --agent2 mcts --size 5
```

图形化界面

要求：设计必要的图形化界面，实现人机对弈功能。

建议工具：

- PyQt / PySide
- Tkinter（Python 内置）
- pygame

功能建议：

- 显示棋盘和棋子
- 支持鼠标点击落子
- 显示当前回合、提子数等信息
- 支持新游戏、悔棋等功能

快速开始

1. 测试基础设施

确保 `dlgo` 模块正常工作：

```
python -c "from dlgo import GameState; g = GameState.new_game(5); print('OK:', g.board
```

2. 开始实现

按照三个小问的顺序，依次实现：

1. 编辑 `agents/random_agent.py` （第一小问）
2. 编辑 `agents/mcts_agent.py` （第二小问）
3. 编辑 `agents/minimax_agent.py` （第三小问，选做）

3. 测试实现

```
# 测试随机 AI
python -c "
from dlgo import GameState
from agents.random_agent import RandomAgent
game = GameState.new_game(5)
agent = RandomAgent()
move = agent.select_move(game)
print('随机 AI 选择:', move)
"

# 测试 MCTS AI
python -c "
from dlgo import GameState
from agents.mcts_agent import MCTSAgent
game = GameState.new_game(5)
agent = MCTSAgent(num_rounds=100)
move = agent.select_move(game)
print('MCTS 选择:', move)
"
```

评分说明

题目一：围棋AI（100分 + 20分选做）

- 第一小问：随机AI（15分）
- 第二小问：MCTS AI（55分）
- 图形化界面（15分）

- 实验报告（15分）
- 第三小问：Minimax AI（选做，20分）

题目二：象棋AI（100分 + 20分选做）

- 第一小问：规则框架与随机AI（45分）
- 第二小问：搜索AI（25分）
- 图形化界面（15分）
- 实验报告（15分）
- 第三小问：扩展功能（选做，20分）

提交要求

提交内容

1. **Python 源码**：包含所有实现的 `.py` 文件
2. **报告**：包含以下内容
    - 设计思路
    - AI 对战结果分析
    - 与 AlphaGo/AlphaZero 的对比思考

报告建议内容

- 算法设计思路和实现细节
- MCTS 优化方法的效果对比
- 不同算法（MCTS vs Minimax）的性能分析
- 图形界面的设计说明
- 测试结果和截图

参考资料

- 《深度学习与围棋》(Deep Learning and the Game of Go)
- AlphaGo 论文：Mastering the game of Go with deep neural networks
- AlphaZero 论文：Mastering the Game of Go without Human Knowledge

常见问题

Q: MCTS 模拟要走多少步?

A: 标准做法是走至终局，但可以限制深度（如 20-30 步）来提升效率。

Q: 如何判断终局?

A: 双方连续 pass 或棋盘填满时终局，具体参见 `dlgo/goboard.py` 中的 `is_over()` 方法。